



B.M.S. COLLEGE OF ENGINEERING **Bengaluru-560019.**

Autonomous College, affiliated to
Visvesvaraya Technological University, Belgaum



Project Work -2 Report on
**“Implementation of RISC-V Instruction Set
Architecture for Embedded Applications”**

Submitted in partial fulfilment of the requirement for completion of
PROJECT WORK -2 [22EC6PWPJ2]

Submitted by

Nikhil H Raju

1BM21EC088

Priyadarshini R

1BM21EC113

Under the guidance of

Mrs. Monika Sharma D

Assistant Professor

BMS College of Engineering

Bengaluru

Academic Year

2023 – 2024

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

BMS COLLEGE OF ENGINEERING

Autonomous college, affiliated to VTU

Bull Temple Road, Bengaluru – 560 019

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING



CERTIFICATE

This is to certified that the Project work -2 entitled **“Implementation of RISC-V Instruction Set Architecture for Embedded Applications”** is a bonafide work carried out by **Nikhil H Raju – 1BM21EC088, Priyadarshini R - 1BM21EC113** submitted in partial fulfilment of the requirement for completion of PROJECT WORK - 2 [22EC6PWPJ2] of Bachelor of Engineering in Electronics and Communication during the academic year 2023-24. The Project Work - 2 report has been approved as it satisfies the academic requirements.

Guide

Mrs. Monika Sharma D

Assistant Professor

Department of ECE
BMS College of Engineering

Head of Department

(Dr. Siddappaji)

Professor and Head
Department of ECE
BMS College of Engineering

(Dr. Muralidhara S)

Principal

BMS College of Engineering

External Viva

Name of the Examiners

Signature with Date

1.

2.

DECLARATION

We, **Nikhil H Raju – 1BM21EC088, Priyadarshini R – 1BM21EC113** hereby declare that the Project Work - 2 entitled **Implementation of RISC-V Instruction Set Architecture for Embedded Applications** is a bonafide work and has been carried out by us under the guidance of **Mrs. Monika Sharma D, Assistant Professor**, Department of Electronics and Communication Engineering, BMS College of Engineering, Bengaluru submitted in partial fulfilment of the requirement for completion of PROJECT WORK - 2 [22EC6PWPJ2] of Bachelor of Engineering in Electronics and Communication during the academic year 2023-24. The Project Work -2 report has been approved as it satisfies the academic requirements in Electronics and Communication engineering, Visvesvaraya Technological University, Belagavi, during the academic year 2023-24.

We further declare that, to the best of our knowledge and belief, this Project Work -2 has not been submitted either in part or in full to any other university.

Place: Bengaluru

Nikhil H Raju – 1BM21EC088

Date:

Priyadarshini R – 1BM21EC113

ACKNOWLEDGEMENTS

We take this opportunity to express our profound gratitude to the respected principal Dr. S. Muralidhara, BMS College of Engineering for providing a congenial environment to work in. Our sincere gratitude to Dr. Siddappaji, Head of the Department, Electronics and Communication Engineering for encouraging and providing opportunity to carry Project Work -2 in the department.

We heartfully thank our guide **Mrs. Monika Sharma D, Assistant Professor** for the guidance and constant encouragement throughout the course of this Project Work -2 without which this project would not be successful.

A number of personalities, in their own capacities have helped us in carrying out this Project Work -2. We would like to take this opportunity to thank them all.

Last but not the least, we thank our friends and family for their encouragement and help in accomplishing the objective of the Project Work -2 work.

- 1) **Nikhil H Raju – 1BM21EC088**
- 2) **Priyadarshini R – 1BM21EC113**

ABSTRACT

In the realm of educational RISC-V processor implementations, existing solutions often present challenges for beginners due to their complexity and limited support for customization and interfacing with FPGA boards. This paper addresses these challenges by proposing a novel approach: the development of a single-cycle 32-bit RISC-V processor optimized for educational environments. Utilizing an Integer set microarchitecture and leveraging an I/O memory-mapped approach, our processor facilitates seamless integration and control of external devices through dedicated memory locations. This design not only simplifies the learning curve for students but also enhances educational value by providing a practical platform for understanding embedded system principles and implementing real-world applications. By emphasizing simplicity, flexibility, and FPGA compatibility, our solution aims to empower educators and learners to explore embedded system design effectively, paving the way for innovation and deeper understanding in the field of computer architecture and digital design. This abstract captures the essence of your research paper, highlighting the innovative aspects of your proposed solution while succinctly outlining its educational and practical implications.

Index Terms—FPGA, RISC-V, CPU Design, Memory Mapped I/O, Verilog

TABLE of CONTENTS

Abstract	
List of figures	5
List of tables	7
List of abbreviations	8
<u>TOPIC</u>	<u>Page No</u>
Chapter 1: Introduction	10
Chapter 2: Literature survey	11
Chapter 3: Problem Analysis & Solution	14
3.1 Problem Definition	
3.2 Proposed Solution	
Chapter 4: Methodology & Implementation	16
4.1 RV32I Base Integer Instruction Set	
4.2 Development And Implementation of RISC V on FPGA	
Chapter 5: Results & Discussion	22
5.1 Simulations of RISC V 32 Microarchitecture	
5.2 Hardware Implementation on Basys 3 FPGA board	
Chapter 6: Future Trends	26
6.1 Conclusion	
6.2 Future Trends	
References	27
Appendix A: Plagiarism Report	
Appendix B	
Appendix C: Research Publication	

List of Figures

Sl.no	FIGURE	Pg.no
1.	Register File	16
2.	Instruction Format	17
3.	Instruction Type, Opcode, And Definition	17
4.	RISC-V Top Level Block	19
5.	Micro Architecture of RV32I ISA Based RISC-V	19
6.	RISC V I/O System Using Memory Mapped I/O	21
7.	I Type Instruction	22
8.	S Type Instruction	23
9.	R Type Instruction	23
10.	B Type Instruction	23
11.	Generated Netlist	24
12.	RISC V I/O System	25
13.	Working Of RISC V I/O System	25

List of tables

Table No.	Description of table	Pg.no
4.1	Summarizing The Control Signals for Each Opcode	18
4.2	Alu Control For R_Type And I_Type Based on Funct3 And Funct7	18

List of abbreviations

Abbreviations	Full Form
IEEE	Institute of Electrical and Electronics Engineers
RISC V	Reduced Instruction Set Computer – V
ARM	Acorn RISC Machine
FPGA	Field-Programmable Gate Array

Chapter 1:

Introduction

Instruction set architectures (ISAs) are standards that define how a processor's hardware should function and interact with its assembly language. An ISA provides all the necessary information to create a processor capable of running machine code correctly and consistently according to the standard. This includes details about instructions, registers, memory access, arithmetic operations, data buses, and more. Although every function of the processor is specified in the standard, the implementation of hardware and circuits is left to the designer's discretion. Most ISAs feature various extensions and variations to meet the requirements of different designs, such as support for multiplication, floating point numbers, diverse data, address and instruction widths, or targeting specific applications like embedded systems, personal computers, and supercomputers.

Historically, commercially successful ISAs have been proprietary, requiring licenses and keeping specific designs and implementations protected by patents and non-disclosure agreements. However, the RISC-V ISA represents a significant shift in the hardware industry as it is the first open-source modern ISA. Developed through a collaboration between the University of California and companies like AMD, Google, Microsoft, IBM, and others, RISC-V has led to the creation and publication of many open-source tools and designs since its initial variants were finalized.

RISC-V is particularly notable not only for its open-source nature but also for its ability to compete with other commercial ISAs from Intel, AMD, and ARM. Unlike many academic designs that are optimized for learning purposes, RISC-V is intended for modern practical use. This open-source nature made RISC-V especially suitable for our project, allowing us to create an open-source CPU using a realistic and practical ISA rather than relying on "toy" or purely academic ISAs.

The customizable nature of the RISC-V core is particularly important for embedded applications. Embedded systems often have specific requirements and constraints, such as power consumption, performance, and size, which can vary widely depending on the application. The flexibility of RISC-V allows designers to tailor the ISA to meet these specific needs, enabling more efficient and effective solutions. This customization capability is a significant advantage in the embedded systems market, where optimizing for particular use cases can lead to substantial improvements in performance and efficiency.

The contribution of our paper is to develop an educational 32-bit RISC-V processor on FPGA that is simple to understand and will support interfacing with external I/O devices for embedded applications. This approach aims to help learners understand the embedded applications of CPUs. The goal of this paper is threefold:

1. To actualize theoretical knowledge through practical experiments.
2. To evaluate and enhance the core's performance.
3. A suite of tools for simulation and compilation
4. To interface the CPU with FPGA, ultrasonic sensor, onboard I/O pins, and IPs.

Existing open-source RISC-V processors, such as PicoRV32 with its integrated complex peripherals and Berkeley's CPUs requiring Chisel expertise, contrast with the industrial-focused SCR1 by Synthacore, highlighting the need for simpler educational RISC-V processor models.

Chapter 2:

Literature survey

1. A. P, A. N, S. S, V. P and A. V, "Implementation of RISC-V Instruction Set Architecture for edge IoT computing platform," 2024 Fourth International Conference on Advances in Electrical, Computing, Communication and Sustainable Technologies (ICAECT), Bhilai, India, 2024, pp. 1-6, doi: 10.1109/ICAECT60202.2024.10468869. [1]

This paper presents a pioneering effort in developing a cost-effective 32-bit processor using the open-source RISC-V (RV32I) ISA, tailored for low-power embedded devices. By detailing the architecture, development process, and validation framework, it underscores the significance of RISC-V in both academic and industrial domains. Through FPGA prototyping and synthesis, the processor achieves impressive performance metrics, showcasing its viability for IoT and embedded applications. Overall, the study makes a compelling case for RISC-V's potential to revolutionize processor design, emphasizing affordability, power efficiency, and simplicity.

2. A. Tiwari, P. Guha, G. Trivedi, N. Gupta, N. Jayaraj and J. Pidanic, "IndiRA: Design and Implementation of a Pipelined RISC-V Processor," 2023 33rd International Conference Radioelektronika (RADIOELEKTRONIKA), Pardubice, Czech Republic, 2023, pp. 1-6, doi: 10.1109/RADIOELEKTRONIKA57919.2023.10109058. [2]

This paper presents a novel approach to enhancing the performance of RISC-V processors through tailored optimizations. By leveraging a five-stage pipeline and fine-tuned arithmetic and logic unit (ALU), along with a dynamic branch predictor, significant improvements in operating frequency are achieved. Implemented using Verilog HDL and validated on FPGA, the proposed processor core demonstrates a remarkable 30% performance boost compared to existing processors. This research highlights the transformative potential of RISC-V architecture in driving computational efficiency and innovation.

3. M. H. Banna Saif, N. U. Sadad and M. N. Islam Mondal, "FPGA Implementation of Educational RISC- V Processor Suitable for Embedded Applications," 2023 International Conference on Electrical, Computer and Communication Engineering (ECCE), Chittagong, Bangladesh, 2023, pp. 1-5, doi: 10.1109/ECCE57851.2023.10101508. [3]

This paper addresses the challenge of teaching computer architecture effectively by proposing a hands-on approach using FPGA technology. Leveraging the open standard RISC-V architecture, the authors develop a 32-bit CPU and accompanying assembler, with support for interfacing in embedded applications. They review existing literature, highlighting gaps in educational methodologies, and present their unique contribution, emphasizing practical implementation and

bridging theory with practice. Through meticulous simulations and comparisons, they demonstrate the effectiveness and originality of their approach. Future research avenues include compiler integration and operating system development to enhance the educational experience.

4. J. -Y. Lai, C. -A. Chen, S. -L. Chen and C. -Y. Su, "Implement 32-bit RISC-V Architecture Processor using Verilog HDL," 2021 International Symposium on Intelligent Signal Processing and Communication Systems (ISPACS), Hualien City, Taiwan, 2021, pp. 1-2, doi: 10.1109/ISPACS51563.2021.9651130. [4]

The paper presents a Verilog-based design of a RISC-V processor supporting the RV32I instruction set architecture, emphasizing its suitability for IoT applications due to features like low power consumption. It discusses prior research on RISC-V implementations, introducing their own processor design and micro-architecture. Simulation results in ModelSim demonstrate successful execution of various instructions, validating the processor's functionality. The study contributes to advancing RISC-V processor development, with future plans including FPGA implementation for further verification.

5. L. Poli, S. Saha, X. Zhai and K. D. McDonald-Maier, "Design and Implementation of a RISC V Processor on FPGA," 2021 17th International Conference on Mobility, Sensing and Networking (MSN), Exeter, United Kingdom, 2021, pp. 161-166, doi: 10.1109/MSN53354.2021.00037. [5]

The paper addresses the growing demand for lightweight RISC-V processors suitable for educational purposes and small FPGAs, aligning with the increasing adoption of RISC-V in IoT and SoC applications. Methodologically, the authors adopt the RV64I extension, selecting seven fundamental instructions to cover essential operations. Each processor component, including instruction memory, register file, ALU, data memory, and control unit, is meticulously designed in System Verilog, prioritizing simplicity and readability. The design philosophy emphasizes straightforwardness, with all instructions executing in a single clock cycle and pipelining omitted for ease of understanding. The control unit utilizes a combinational approach, and concise System Verilog test benches are employed to validate each component. Demonstrating Turing completeness, the paper maps Brainfuck language instructions to RISC-V equivalents, showcasing their functional equivalence. The physical implementation is executed on the Basys 3 Artix-7 FPGA Trainer Board using Vivado Design Suite, facilitating rapid prototyping and experimentation. Evaluation metrics reveal low FPGA resource utilization and strong performance, especially in terms of DMIPs/MHz, surpassing other common CPUs.

6. T. McGrew, E. Schonauer and P. Jamieson, "Framework and Tools for Undergraduates Designing RISC-V Processors on an FPGA in Computer Architecture Education," 2019 International Conference on Computational Science and Computational Intelligence (CSCI), Las Vegas, NV, USA, 2019, pp. 778-781, doi: 10.1109/CSCI49370.2019.00148. [6]

The paper advocates for using RISC-V architecture in undergraduate computer engineering education, citing its relevance, open accessibility, and educational benefits. It proposes a framework for students to build RISC-V processors on FPGAs, emphasizing hands-on learning. The authors discuss previous teaching approaches and describe student implementations with extensions like cache and multicore systems. They highlight the pedagogical value of project-based learning and suggest further exercises to deepen understanding. Overall, the paper promotes RISC-V as an ideal educational processor choice and provides a structured approach for educators and students.

7. E. Gür, Z. E. Sataner, Y. H. Durkaya and S. Bayar, "FPGA Implementation of 32-bit RISC-V Processor with Web-Based Assembler-Disassembler," 2018 International Symposium on Fundamentals of Electrical Engineering (ISFEE), Bucharest, Romania, 2018, pp. 1-4, doi: 10.1109/ISFEE.2018.8742406. [7]

The paper introduces a new project focused on crafting a compact, 32-bit RISC-V processor using Verilog and deploying it on a Cyclone IV 4CE115 FPGA device. Alongside this development, the authors present user-friendly web tools for assembling and disassembling code, enhancing the accessibility of the processor. These tools, built with modern web technologies, simplify the generation and translation of machine code for the RISC-V processor. The processor itself operates on the RV32I instruction set, boasting full functionality with 37 instructions while using hardware resources judiciously, consuming around 43.7% of the FPGA device. Impressively, it achieves a maximum frequency of 150MHz without requiring any timing constraints. The paper systematically explores the RV32I instruction set, detailing its key features, register organization, and instruction types. It then delves into the design and micro-architecture of the RISC-V processor, emphasizing simplicity and efficiency. The methodology adopts a single-cycle architecture, dissecting the processor's modules and micro-architecture stages. Simulation and FPGA implementation are comprehensively covered, showcasing the utility of the web-based tools in easing programming tasks. The paper offers both theoretical insights and practical demonstrations of the RISC-V processor's design and functionality.

8. D. K. Dennis et al., "Single cycle RISC-V micro architecture processor and its FPGA prototype," 2017 7th International Symposium on Embedded Computing and System Design (ISED), Durgapur, India, 2017, pp. 1-5, doi: 10.1109/ISED.2017.8303926. [8]

This paper presents the development of a 32-bit processor based on the RISC-V ISA, tailored for low-cost embedded devices. It's designed as a single-core, in-order, non-bus based processor implemented in Verilog HDL and prototyped on an FPGA board. Achieving a maximum frequency of 32MHz with a power consumption of 7.9mW, it also introduces a RISC-V development framework and automated test suites. The architecture includes modules for instruction fetching, decoding, control logic, register bank, ALU, and memory control. Future goals involve integrating I/O bus, adding multi-level cache, incorporating interrupts, and a floating-point co-processor, highlighting RISC-V's potential in IoT and embedded applications.

Chapter 3:

Problem Analysis & Solution

3.1 Problem Definition

In the current scenario of RISC-V processor implementations for educational purposes, several challenges persist. First, existing implementations like PicoRV32, while beneficial for their simplicity, may still be too complex for beginners due to their integration of peripherals and the intricacies of FPGA implementation. PicoRV32, developed by Claire Wolf, is a compact RISC-V CPU for FPGA use with support for peripherals like UARTs and GPIO controllers, suitable for versatility but potentially complex for beginners.

Berkeley's CPUs (Rocket, BOOM, Sodor) are advanced RISC-V implementations requiring expertise in languages like Chisel, tailored for academic and research applications.

SCR1 by Synthacore is aimed at industrial use, featuring advanced features like pipelining and AXI4/AHB-Lite interfaces, which may be challenging for educational settings focused on simplicity and accessibility.

Secondly, many educational platforms designed around RISC-V prioritize simplicity but may lack comprehensive support for extending functionality or interfacing with specific FPGA boards like Basys 3 and Arty. This limitation can hinder students who seek to explore advanced applications or require customization beyond basic configurations.

Lastly, while there are community-driven efforts to create customizable RISC-V implementations for FPGA boards, the availability of comprehensive tutorials and documentation to support hands-on learning and experimentation remains inconsistent. This gap can make it challenging for educators and learners to effectively utilize these platforms for practical educational goals.

Addressing these challenges could enhance the accessibility and educational value of RISC-V processors in FPGA-based learning environments, fostering greater understanding and innovation among students and developers.

3.2 Proposed Solution

The development of a single-cycle 32-bit RISC-V processor employing an Integer set microarchitecture with a focus on simplicity and robust interfacing capabilities for FPGA boards is essential. This processor will implement an I/O memory-mapped approach, allowing effective control and monitoring of I/O devices through dedicated memory locations within the system.

In this architecture, the processor will integrate specialized decoding logic designed to monitor address and control signals. When a 'store word' ('sw') instruction is executed, indicating a write operation, the decoding logic will intercept and route data to specific memory addresses allocated for I/O devices. This approach, known as Memory Mapped I/O, ensures efficient communication between the processor and peripherals without the need for additional complex interfaces or protocols.

Key features of this design include:

- **Simplified Integration:** By utilizing memory-mapped I/O, the processor streamlines the integration of I/O devices into the system. Each device is assigned a unique memory location, facilitating straightforward data transfer between the processor and peripherals.
- **Flexibility and Scalability:** The architecture supports easy addition of new I/O devices by simply defining new memory-mapped addresses and updating the decoding logic. This scalability allows for rapid prototyping and experimentation with various input and output configurations.
- **Educational Value:** Designed with educational objectives in mind, this processor provides a clear illustration of core RISC-V principles and practical implementation of memory-mapped I/O techniques. Students gain hands-on experience in system design, understanding how to control and interface with external devices using a simplified yet powerful processor architecture.
- **FPGA Compatibility:** Tailored for FPGA boards such as Basys 3 and Arty, the design ensures compatibility and ease of implementation. FPGA constraints can be readily applied to manage timing, resource allocation, and interface requirements, making it suitable for both educational projects and real-world embedded applications.

By focusing on these aspects, the proposed solution aims to empower students and developers alike to explore embedded system design effectively, leveraging the power and versatility of FPGA technology in a straightforward and educational manner.

Chapter 4:

Methodology & Implementation

4.1 RV32I Base Integer Instruction Set

A. Characteristics

RV32I has four core instruction formats (R/I/S/U). All are a fixed 32 bits in length and must be aligned on a four-byte boundary in memory. Immediate instruction type is split into five different immediate types (I/S/B/U/J). Sign extension uses the left-most bit in instruction. There are a further two variants of the instruction formats (SB/UJ) based on the handling of immediate values. It contains 47 different and unique instructions. RV32I contains 47 unique instructions, though an implementation might cover the eight SCALL/SBREAK/RD instructions with a single SYSTEM hardware instruction that always traps, reducing hardware instruction count to 40 total.

B. Register organization

There are 31 general-purpose registers x1–x31, which hold integer values. Register x0 is hardwired to the constant 0. Standard software calling convention uses register x1 to hold the return address on a call.

Name	Register Number	Usage
zero	x0	Constant value 0
ra	x1	Return address
sp	x2	Stack pointer
gp	x3	Global pointer
tp	x4	Thread pointer
t0-2	x5-7	Temporaries
s0/fp	x8	Saved register / Frame pointer
s1	x9	Saved register
a0-1	x10-11	Function arguments / return values
a2-7	x12-17	Function arguments
s2-11	x18-27	Saved registers
t3-6	x28-31	Temporaries

Fig. 1. Register File

C. Instruction formats

It is also possible to encode an ISA with 16 registers but it lowers the efficiency and increases the instruction count. To avoid intermediate instruction sizes RV32I encoded with 32 registers.

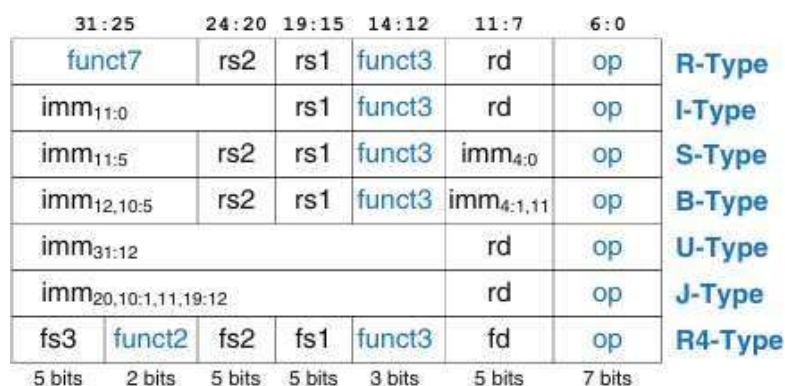


Fig. 2. Instruction Format

D. Instructions

RV32I instructions are classified based on the following instruction bits; instruction[30], function3[14:12], Opcode[6:0]. It has 7 different instruction category and every category is explained in Fig. 3.

CATEGORY	OPCODE	INSTRUCTION	DEFINITION
Integer Register-Immediate Instructions (I-Type)	0010011	addi, slti, sltiu, ori, xori, andi, slli, srli, srai	addi rd, rs1, imm Adds rs1 and imm and then it writes the answer to the rd
Integer Register-Register Operations (R-Type)	0110011	add, sub, sll, slt, sltu, xor, srl, sra, or, and	add rd, rs1, rs2 Adds rs1 and rs2 and then it writes the answer to the rd
Integer Computational Instructions (U-Type)	0110111 0010111	lui auipc	lui rd,imm Load immediate into upper bits of word
Unconditional Jumps (UJ, I-Type)	1101111	jal, jalr	jal rd,imm Jumps to adress and place return adress to the rd
Conditional Branches (SB-Type)	1100011	beq, bne, blt, bltu, bge, bgeu	beq rs1,rs2,imm If rs1 and rs2 is equal, branches into the immediate value adress
Load Instruction (I-Type)	0000011	lw, lh, lb, lhu, lbu	lw rd,rs1,imm Adds rs1 and imm and then writes the corresponding value from memory to rd
Store Instruction (S-Type)	0100011	sw, sh, sb	sw rs1,rs2,imm Adds rs1 and imm and then writes the value of rd to the corresponding adress in memory

Fig. 3. Instruction Type, Opcode and Definition

E. Control Unit and ALU Unit

The control unit of a processor serves as the orchestrator of operations within the CPU, crucially decoding instructions fetched from memory to generate precise control signals. Its primary role is to interpret each instruction's opcode, determining the type of operation (such as arithmetic, memory access, or control

flow) and subsequently generating signals that direct the processor's functional units accordingly. These signals manage activities like reading and writing from registers, selecting operations for the ALU, controlling memory access, and managing the program counter for branching and jumping. Whether implemented through a finite state machine, combinational logic, or microcode, the control unit ensures that instructions are executed in the correct sequence and timing, optimizing the CPU's performance, power efficiency, and overall functionality. Its accuracy and efficiency are fundamental to the smooth operation and capability of modern processors across a wide range of computing tasks.

The Arithmetic Logic Unit (ALU) within a processor performs essential arithmetic and logical operations on binary data. It handles tasks such as addition, subtraction, bitwise AND, OR, XOR, and shifts (both logical and arithmetic). The ALU is responsible for executing these operations swiftly and efficiently, using specialized circuits to manipulate binary numbers and produce results that are vital for computational tasks. Its capabilities include setting flags for conditions like carry or overflow, performing data encryption, and enabling bit-level operations crucial for manipulating and processing data within the CPU. The ALU's efficiency directly impacts the overall performance and versatility of the processor in executing diverse computing tasks.

Table summarizing the control signals for each opcode:

Opcode	reg_write	mem_write	ALUSrc	ResultSrc	PCsrc	ALUcontrol	ImmSrc
R_TYPE	1	0	0	00	0	See funct3	-
LOAD	1	0	1	01	0	0000	00
STORE	0	1	1	-	0	0000	01
BRANCH	0	0	0	-	1	0001	10
JAL	1	0	0	10	1	-	11
JALR	1	0	1	10	1	-	00
I_TYPE	1	0	1	00	0	See funct3	00

Table 4.1 Summarizing the control signals for each opcode

Detailed ALU Control for R_TYPE and I_TYPE based on funct3 and funct7:

funct3	funct7	ALUcontrol	Description
000	0000000	0000	ADD
000	0100000	0001	SUB
111	-	0010	AND
110	-	0011	OR
001	-	0100	SLL
101	-	0105	SRL
100	-	0117	XOR
010	-	1000	SLT

Table 4.2 ALU Control for R_TYPE and I_TYPE based on funct3 and funct7

4.2 Development And Implementation of RISC V on FPGA

A. Overview

The processor operates using a single-cycle data path with concurrent modules controlled by specific control signals. After fetching an instruction, the decode and control stage analyzes the instruction to determine the progression of operations. Key modules include the register file containing 32 registers (with x0 always holding 0) and the ALU handling arithmetic operations. Branch instructions are

evaluated for conditions by the ALU without calculating jump addresses directly.

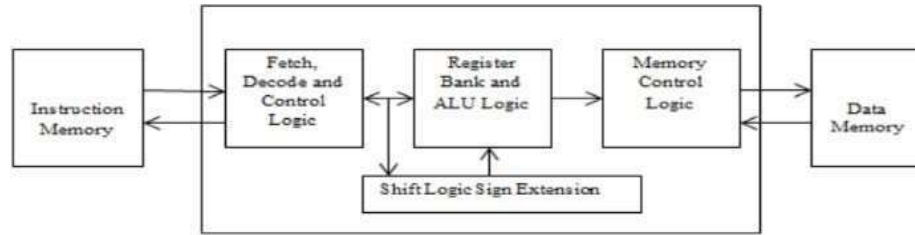


Fig. 4. RISC V Top Level Block

B. Microarchitecture

In the micro-architecture, the fetch stage calculates the current and next program counter (PC) values, adjusting the PC based on jump or branch instructions. During decode, control signals are generated from instruction fields (opcode, funct3) to direct module operations. The ALU performs computations based on these signals, and memory operations transfer data between registers and memory based on load (LW, LH, LB, LBU, LHU) and store (SW, SH, SB) instructions. This structured approach ensures efficient instruction execution within a single cycle, guided by control signals derived from instruction decoding.

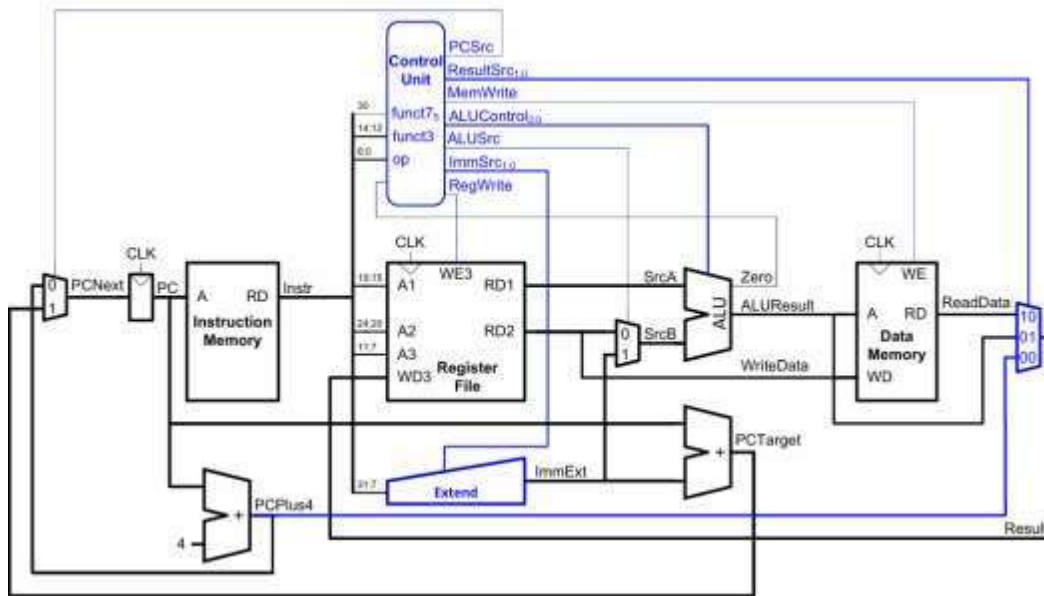


Fig. 5. Micro architecture of RV32I ISA based RISC-V

C. RISC-V Toolchain

The RISC-V toolchain encompasses a suite of software tools tailored for developing applications on RISC-V architecture-based systems. This toolchain includes an assembler (riscv32-unknown-elf-as) for translating human-readable RISC-V assembly code into machine code, a linker (riscv32-unknown-elf-ld) for combining object files into executable programs, and a compiler (riscv32-unknown-elf-gcc) capable of translating high-level languages like C into RISC-V assembly or machine code. Additionally, a debugger (riscv32-unknown-elf-gdb) is available for debugging RISC-V programs, enabling developers to inspect and modify program state during execution. RISC-V assembly language, used in conjunction with the toolchain, allows programmers to write low-level code directly for RISC-V processors, leveraging mnemonic instructions, registers, and memory access commands to control hardware behavior efficiently and effectively.

D. Interfacing

The integration of a 32-bit RISC-V processor with an ultrasonic sensor, two switches, and a bank of six LEDs on the Basys 3 FPGA board utilizes a memory-mapped approach for efficient data handling. Input signals from the ultrasonic sensor and switches are captured and stored in dedicated memory locations, facilitated by the processor's assembly language instructions. This setup ensures precise control over data flow and interaction with external devices, enabling real-time data processing and response. Detailed constraints have been meticulously defined to optimize resource utilization and ensure compatibility, culminating in the successful generation of a bitstream for FPGA deployment and thorough testing.

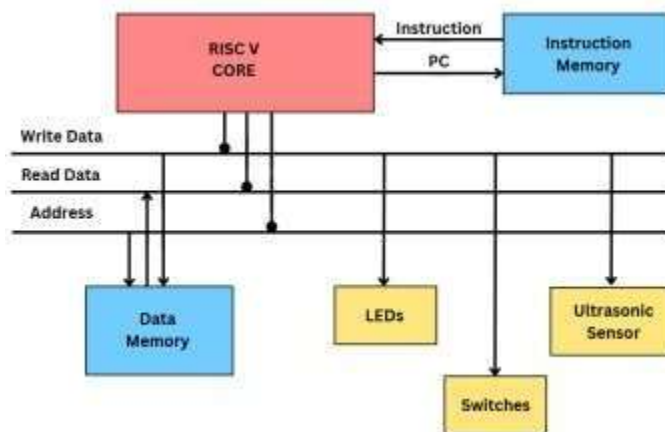


Fig. 6. RISC V I/O system using memory mapped approach

E. Testing and Verification

Simulation and verification of the design are carried out using Xilinx Vivado. Comprehensive testbenches simulate different instruction scenarios to validate each module's functionality. Waveform viewers in Vivado are used for visually inspecting signal transitions, ensuring the correctness of the design. Functional testing verifies the proper execution of each instruction type and ensures the correct

interaction between data path components and control logic. Edge cases such as overflow, underflow, and invalid instructions are thoroughly tested to ensure robust performance. Performance metrics, including clock cycle time, are measured to confirm that each instruction completes in a single cycle.

For embedded applications, the Vivado design flow is utilized post-simulation to synthesize the design. The synthesized design is converted into a bitstream file (.bit), which is then deployed onto an FPGA for testing. This bitstream generation and deployment process ensures the processor's functionality in real-world scenarios by running embedded applications. Real-time testing on the FPGA verifies the processor's performance in practical settings, confirming its reliability and effectiveness.

Chapter 5:

Results & Discussion

In this project, a single-cycle 32-bit RISC-V processor with an I-type microarchitecture is implemented using Verilog HDL. The design and development process involve several critical components and steps to ensure the successful creation of the processor. Key specifications include the implementation of I-type, B - type, S - type, R - type, J - types instructions of the RISC-V ISA, an ALU, register file, instruction memory, and data memory, all managed by a control unit that governs data and control signal flow.

We simulated our design using Vivado software to verify our designs.

5.1 Simulations of RISC V 32 Microarchitecture

A. I type Instructions

Instruction Details: Address: 0x1000, Label: L7, Instruction: lw x6, -4(x9), Immediate (imm[11:0]): 111111111100 (binary) or FFC (hex), Source Register (rs1): 01001 (x9), Function3 (funct3): 010, DestinationRegister (rd): 00110 (x6), Opcode (op): 0000011.
Machine Language Encoding: FFC4A303.

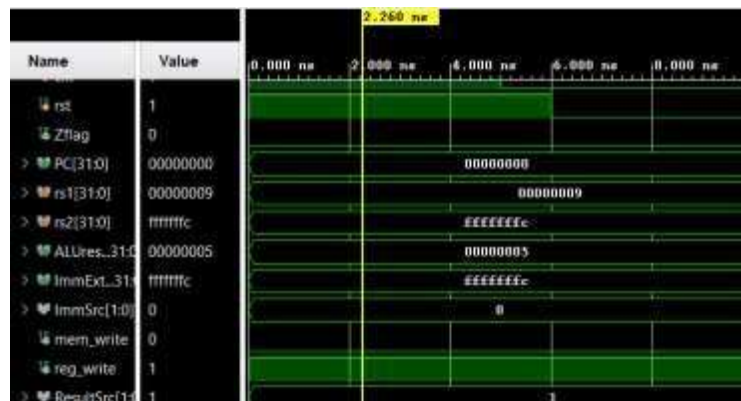


Fig.7 I type Instruction

B. S type Instructions

Instruction Details: Address: 0x1004, Instruction: sw x6, 8(x9), Immediate (imm[11:0]): 0000000001000(binary) or 008 (hex), Source Register (rs1): 01001 (x9), Source Register 2 (rs2): 00110 (x6), Function3 (funct3): 010, Opcode (op): 0100011.
Machine Language Encoding: 0064A423.

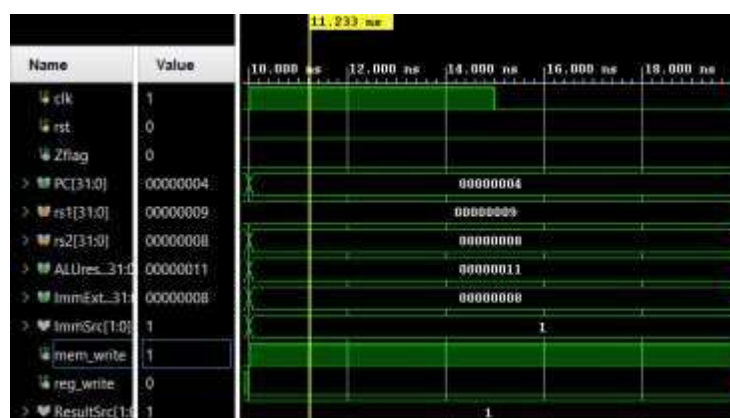


Fig.8 S type Instruction

C. R type Instructions

Instruction Details: Address: 0x1008, Instruction: or x4, x5, x6, Source Register (rs1): 00101 (x5), Source Register (rs2): 00110 (x6), Function3 (funct3): 110, Destination Register (rd): 00100 (x4), Opcode (op): 0110011.

Machine Language Encoding: 0062E233.

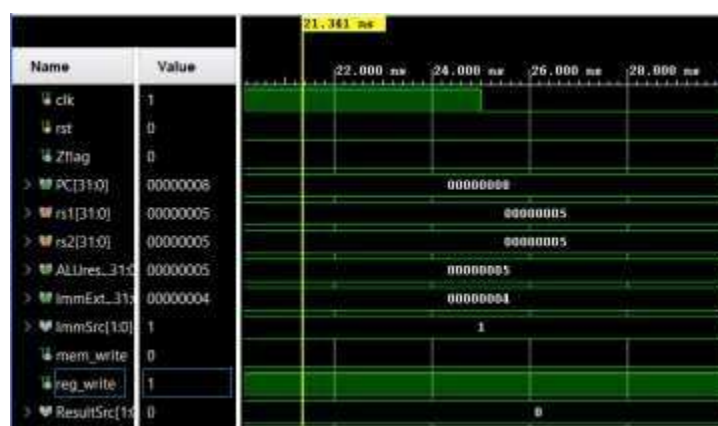


Fig.9 R type Instruction

D. B type Instructions

Instruction Details: Address: 0x100C, Instruction: beq x4, x4, L7, Immediate (imm[12,10:5]): 111111100100 (binary) or FE4 (hex), Source Register (rs1): 00100 (x4), Source Register (rs2): 00100 (x4), Function3 (funct3): 000, Opcode (op): 1100011.

Machine Language Encoding: FE420AE3.

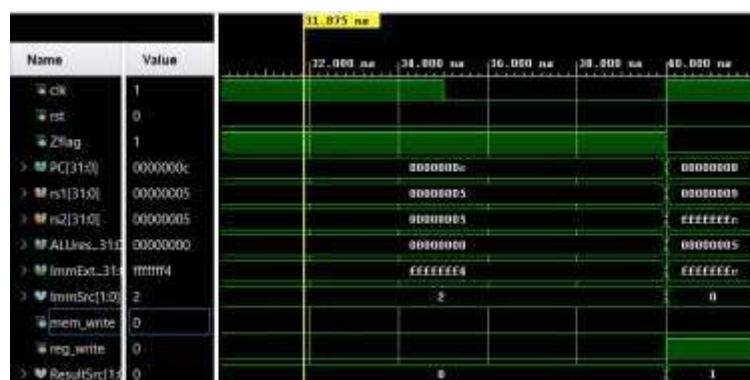


Fig.10 B type Instruction

5.2 Hardware Implementation on Basys 3 FPGA board

Implemented on the Basys 3 FPGA board, the integration of a 32-bit RISC-V processor with an ultrasonic sensor, two switches, and six LEDs utilized a memory-mapped approach for efficient data handling. Assembly language instructions managed input signals, ensuring real-time processing and precise control over LED outputs. Thorough testing validated functionality and optimization, culminating in successful FPGA deployment. The results are given below.

A. Synthesis

For synthesis, the design of the 32-bit RISC-V processor with integrated ultrasonic sensor, switches, and LEDs on the Basys 3 FPGA board was successfully converted into hardware using Vivado. Constraints were meticulously applied to optimize resource utilization, ensuring efficient implementation and reliable performance of the system.

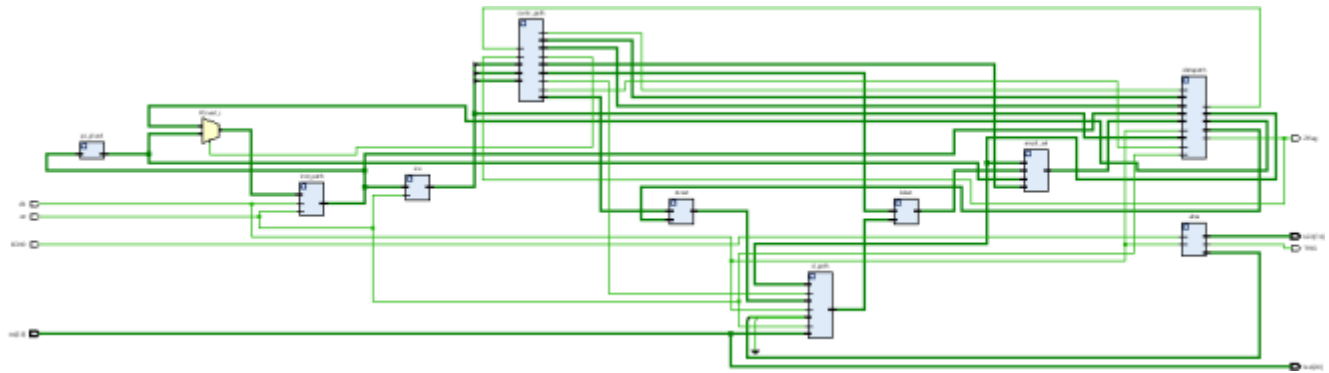


Fig.11 Generated Netlist

B. Implementation

For implementation in Vivado, the design process involved synthesizing the 32-bit RISC-V processor's Verilog HDL code, configuring it with system constraints, and generating a bitstream for deployment onto the Basys 3 FPGA board. Rigorous testing and verification were conducted to ensure functionality and performance adherence to specifications. Approximately 31% of the FPGA board's Lookup Tables (LUTs) were utilized during the implementation of the 32-bit RISC-V processor design in Vivado. Static Timing Analysis (STA) was conducted to analyze and verify the timing performance of the 32-bit RISC-V processor design implemented on the Basys 3 FPGA board using Vivado.



Fig.12 RISC V I/O system

C. Bitstream Generation and testing

The generated bitstream was successfully deployed and tested on the Basys 3 FPGA board, demonstrating functional integration with ultrasonic sensors, switches, and LEDs.

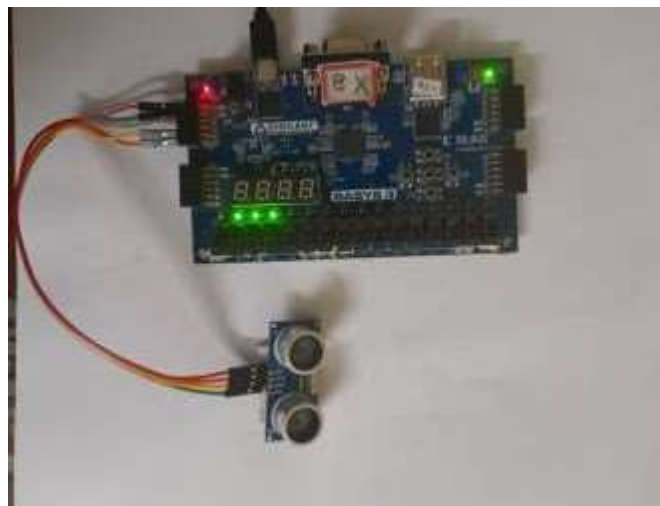


Fig.13 Working of RISC- V I/O system

Chapter 6:

Future Trends and Conclusion

6.1 Conclusion

The implementation of the single-cycle 32-bit RISC-V processor with an Integer set microarchitecture and I/O memory-mapped approach has successfully addressed key challenges in educational RISC-V processor implementations. By prioritizing simplicity, robust interfacing capabilities for FPGA boards like Basys 3 and Arty, and comprehensive educational support, the processor enables effective integration and control of I/O devices through dedicated memory locations. Testing and validation have confirmed the processor's ability to facilitate straightforward data transfer and scalability for additional peripherals, enhancing both educational value and practical application in FPGA-based learning environments. This approach not only supports hands-on learning of core RISC-V concepts but also promotes innovation and exploration in embedded system design among students and developers.

6.2 Future Trends

Future trends for RISC-V processors for embedded applications include:

1. **Increased Customization and Extension:** There will be a growing demand for RISC-V processors that allow easy customization and extension of functionalities tailored to specific educational needs. This trend will focus on providing modular architectures that support the addition of new instructions, peripherals, and interfaces without requiring deep hardware design expertise.
2. **Integration with Embedded Applications:** Future RISC-V processors will continue to integrate with various embedded applications, leveraging FPGA capabilities to support real-time processing, control systems, and sensor interfacing. This integration will enable students to explore practical implementations such as industrial automation, automotive electronics, and smart agriculture within FPGA-based educational projects.
3. **Enhanced Security Features:** With increasing concerns over cybersecurity, future RISC-V processors for educational use may incorporate enhanced security features. This could include hardware-level security mechanisms, cryptographic accelerators, and secure boot capabilities to protect data and ensure system integrity in embedded applications.
4. **Optimization for Low-Power Applications:** There will be a focus on optimizing RISC-V processors for low-power applications, suitable for battery-operated embedded systems and IoT devices. Future designs may emphasize energy-efficient architectures and power management techniques, enabling students to explore sustainable and environmentally friendly embedded solutions.

These future trends underscore the versatile applications of RISC-V processors in educational settings, offering students and educators opportunities to explore advanced technologies and prepare for careers in embedded system development, IoT deployment, and real-world embedded applications.

REFERENCES

- [1] A. P, A. N, S. S, V. P and A. V, "Implementation of RISC-V Instruction Set Architecture for edge IoT computing platform," 2024 Fourth International Conference on Advances in Electrical, Computing, Communication and Sustainable Technologies (ICAECT), Bhilai, India, 2024, pp. 1- 6, doi: 10.1109/ICAECT60202.2024.10468869.
- [2] A. Tiwari, P. Guha, G. Trivedi, N. Gupta, N. Jayaraj and J. Pidanic, "IndiRA: Design and Implementation of a Pipelined RISC-V Processor," 2023 33rd International Conference Radioelektronika (RADIOELEKTRONIKA), Pardubice, Czech Republic, 2023, pp. 1-6, doi: 10.1109/RADIOELEKTRONIKA57919.2023.10109058.
- [3] M. H. Banna Saif, N. U. Sadad and M. N. Islam Mondal, "FPGA Implementation of Educational RISC- V Processor Suitable for Embedded Applications," 2023 International Conference on Electrical, Computer and Communication Engineering (ECCE), Chittagong, Bangladesh, 2023, pp. 1-5, doi: 10.1109/ECCE57851.2023.10101508.
- [4] J. -Y. Lai, C. -A. Chen, S. -L. Chen and C. -Y. Su, "Implement 32-bit RISC-V Architecture Processor using Verilog HDL," 2021 International Symposium on Intelligent Signal Processing and Communication Systems (ISPACS), Hualien City, Taiwan, 2021, pp. 1-2, doi: 10.1109/ISPACS51563.2021.9651130.
- [5] L. Poli, S. Saha, X. Zhai and K. D. McDonald-Maier, "Design and Implementation of a RISC V Processor on FPGA," 2021 17th International Conference on Mobility, Sensing and Networking(MSN), Exeter, United Kingdom, 2021, pp. 161-166, doi: 10.1109/MSN53354.2021.00037.
- [6] T. McGrew, E. Schonauer and P. Jamieson, "Framework and Tools for Undergraduates Designing RISC-V Processors on an FPGA in Computer Architecture Education," 2019 International Conference on Computational Science and Computational Intelligence (CSCI), Las Vegas, NV, USA, 2019, pp. 778-781, doi: 10.1109/CSCI49370.2019.00148.
- [7] E. Gür, Z. E. Sataner, Y. H. Durkaya and S. Bayar, "FPGA Implementation of 32-bit RISC-V Processor with Web-Based Assembler-Disassembler," 2018 International Symposium on Fundamentals of Electrical Engineering (ISFEE), Bucharest, Romania, 2018, pp. 1-4, doi: 10.1109/ISFEE.2018.8742406.
- [8] D. K. Dennis et al., "Single cycle RISC-V micro architecture processor and its FPGA prototype," 2017 7th International Symposium on Embedded Computing and System Design (ISED), Durgapur, India, 2017, pp. 1-5, doi: 10.1109/ISED.2017.8303926.

APPENDIX A:

Plagiarism Report

Page 1

PLAGIARISM SCAN REPORT

Report Generation Date: 11-07-24

Words: 5665

Characters: 30185

Excluded URL : N/A

3%
Plagiarism

97%
Unique

1
Plagiarized Sentences

38
Unique Sentences

Content Checked for Plagiarism

Instruction set architectures (ISAs) are standards that define how a processor's hardware should function and interact with its assembly language. An ISA provides all the necessary information to create a processor capable of running machine code correctly and consistently according to the standard. This includes details about instructions, registers, memory access, arithmetic operations, data buses, and more. Although every function of the processor is specified in the standard, the implementation of hardware and circuits is left to the designer's discretion. Most ISAs feature various

APPENDIX B:

APPENDIX C:

